

Pruebas de software

Implementacion pruebas de integracion

Nombre del proyecto: EcoSense

Integrantes:

- Billy Martinez
- Bastian Lagos

Fecha de ejecucion: 27/05/2026

Estrategia de implementacion

La estrategia de integracion seleccionada para EcoSense es hibrida. El sistema combina servicios de dominio puros con servicios de aplicacion que coordinan repositorios, clientes externos y efectos laterales.

Las pruebas se ejecutan a nivel de servicio/modulo. No se prueba la interfaz grafica, no se usa emulador Android y no se usan webdrivers. Los puntos de entrada son GrupoApplicationService y RecyclingApplicationService.

Firebase, Storage y la cola se modelan mediante puertos y dobles in-memory controlados por el test, permitiendo evidencia reproducible sin red.

Componentes bajo prueba

Componente	Rol	Doble usado
GrupoApplicationService	Orquesta grupos	Repositorios in-memory y cola fake
RecyclingApplicationService	Orquesta reciclaje y recompensas	Storage fake y repositorios in-memory
GrupoService / RankingService	Reglas puras de dominio	Instancias reales
EventPublisher	Cola o job programado	RecordingEventPublisher

Diseno de pruebas de integracion

IT-01 Creacion exitosa de grupo publico (Camino Correcto)

Integracion: GrupoApplicationService <-> UsuarioRepositoryPort <-> GrupoRepositoryPort <-> EventPublisher

Precondiciones: Repositorio de usuarios con el usuario U001 registrado. Repositorio de grupos vacio. Publicador de eventos RecordingEventPublisher sin mensajes previos.

Entradas: crearGrupo(creadorId = "U001", nombre = "EcoLab", descripcion = "Grupo de reciclaje", tipo = PUBLICO)

Paso	Descripcion
1	El servicio de aplicacion valida que el creador exista en el repositorio de usuarios.
2	Se consulta el repositorio de grupos para descartar conflicto por nombre duplicado.
3	Se invoca GrupoService para crear la entidad de dominio con el creador como ADMINISTRADOR.
4	Se persiste el grupo en el repositorio y se actualiza el groupId del usuario creador.
5	Se publica el evento GROUP_CREATED como efecto lateral equivalente a cola o job interno.

Resultados esperados: El resultado es ResultadoCreacion.Exitoso. La BD in-memory contiene el grupo creado, el usuario U001 queda enlazado al nuevo grupo y la cola registra un unico evento GROUP_CREATED.

IT-02 Rechazo por nombre de grupo duplicado (Camino de Error: 409 Conflict)

Integracion: GrupoApplicationService <-> GrupoRepositoryPort <-> UsuarioRepositoryPort <-> EventPublisher

Precondiciones: Usuario U001 registrado. Repositorio de grupos con un grupo preexistente llamado EcoLab. Cola de eventos vacia antes de ejecutar el caso.

Entradas: crearGrupo("U001", "EcoLab", "Duplicado", PUBLICO)

Paso	Descripcion
1	El servicio valida la existencia del usuario creador.
2	Se consulta el repositorio de grupos por nombre antes de crear una nueva entidad.
3	El repositorio retorna un grupo existente con el mismo nombre.
4	El flujo se corta antes de persistir cambios o publicar eventos.

Resultados esperados: El resultado es ResultadoCreacion.Error con mensaje "Ya existe un grupo con ese nombre". La BD mantiene solo el grupo original, el usuario sigue sin grupo asignado y la cola queda vacia.

IT-03 Union exitosa a grupo publico (Camino Correcto)

Integracion: GrupoApplicationService <-> GrupoService <-> GrupoRepositoryPort <-> UsuarioRepositoryPort <-> EventPublisher

Precondiciones: Usuario U002 sin grupo y con 30 puntos. Grupo publico G001 existente con 100 puntos acumulados. Repositorios in-memory cargados con esos datos.

Entradas: unirseAGrupo(usuariold = "U002", grupold = "G001")

Paso	Descripcion
1	El servicio carga usuario y grupo desde los repositorios.
2	Se reconstruye el estado de dominio en GrupoService con los grupos disponibles.
3	GrupoService aplica la regla de union a grupo publico.
4	Se guarda el usuario con grupold G001 y se actualiza el grupo con el nuevo miembro.
5	El puntaje del usuario se suma al puntaje total del grupo y se publica GROUP_JOINED.

Resultados esperados: El resultado es ResultadoUnion.Exitoso. El usuario U002 queda asociado a G001, el grupo contiene al nuevo miembro, el puntaje grupal sube de 100 a 130 y la cola contiene GROUP_JOINED.

IT-04 Intento de union a grupo inexistente (Camino de Error: 404 Not Found)

Integracion: GrupoApplicationService <-> GrupoRepositoryPort <-> UsuarioRepositoryPort <-> EventPublisher

Precondiciones: Usuario U002 registrado. Repositorio de grupos vacio. Cola sin eventos.

Entradas: unirseAGrupo("U002", "G404")

Paso	Descripcion
1	El servicio recupera el usuario desde el repositorio.
2	Se busca el grupo G404 en el repositorio de grupos.
3	El repositorio retorna null, por lo que se aborta el flujo de dominio.

Resultados esperados: El resultado es ResultadoUnion.Error con mensaje "El grupo no existe". El usuario permanece sin grupo, la BD de grupos sigue vacia y no se publican mensajes.

IT-05 Creacion exitosa de solicitud de reciclaje (Camino Correcto)

Integracion: RecyclingApplicationService <-> ImageStorageClient <-> RecyclingRequestRepositoryPort <-> UsuarioRepositoryPort <-> EventPublisher

Precondiciones: Usuario U010 existente. Repositorio de solicitudes vacio. Cliente storage fake configurado para retornar una URL valida. Cola de eventos vacia.

Entradas: submitRequest("U010", "Plastico", 2.5, byteArrayOf(1, 2, 3), "Botellas PET")

Paso	Descripcion
1	El servicio valida que el usuario exista, que el material no venga vacio y que la cantidad sea positiva.
2	Se invoca ImageStorageClient para subir la imagen de evidencia.
3	Con la URL retornada se crea la solicitud REQ-1 en estado PROCESSING.
4	Se persiste la solicitud en el repositorio y se agrega su id al historial del usuario.
5	Se publica RECYCLING_VALIDATION_REQUESTED para representar el job posterior de validacion.

Resultados esperados: La solicitud queda persistida con estado PROCESSING, URL de storage, descripcion y cantidad correctas. El historial del usuario incluye REQ-1 y la cola contiene RECYCLING_VALIDATION_REQUESTED.

IT-06 Timeout del storage al crear solicitud (Camino de Error: Timeout / 500 externo)

Integracion: RecyclingApplicationService <-> ImageStorageClient <-> RecyclingRequestRepositoryPort <-> UsuarioRepositoryPort <-> EventPublisher

Precondiciones: Usuario U010 existente. Repositorio de solicitudes vacio. Cliente storage fake configurado para lanzar RuntimeException("timeout al subir imagen").

Entradas: submitRequest("U010", "Vidrio", 1.2, byteArrayOf(9), null)

Paso	Descripcion
1	El servicio valida los datos basicos y llama al cliente externo de storage.
2	El cliente storage simula un timeout antes de retornar URL.
3	La excepcion se propaga como Result.failure y se detiene el flujo.
4	No se persisten solicitudes, no se modifica el historial y no se publica job de validacion.

Resultados esperados: La operacion falla con mensaje "timeout al subir imagen". El repositorio de solicitudes queda vacio, el historial del usuario no cambia y la cola queda vacia.

IT-07 Canje exitoso de recompensa (Camino Correcto)

Integracion: RecyclingApplicationService <-> RecyclingRequestRepositoryPort <-> UsuarioRepositoryPort <-> EventPublisher

Precondiciones: Usuario U020 con 10 puntos. Solicitud REQ-7 existente, asociada a U020 y en estado REWARD. Cola de eventos vacia.

Entradas: redeemReward(requestId = "REQ-7", userId = "U020", rewardPoints = 25)

Paso	Descripcion
1	El servicio recupera la solicitud y valida que exista.
2	Se recupera el usuario y se comprueba que la solicitud le pertenezca.
3	Se actualiza la solicitud a estado REEDEMED y se registra reward = 25.
4	Se suman 25 puntos al usuario y se publica REWARD_REEDEMED.

Resultados esperados: La solicitud REQ-7 queda en REEDEMED, el reward queda en 25, el usuario sube de 10 a 35 puntos y la cola registra REWARD_REEDEMED.

IT-08 Canje de recompensa inexistente (Camino de Error: 404 Not Found)

Integracion: RecyclingApplicationService <-> RecyclingRequestRepositoryPort <-> UsuarioRepositoryPort <-> EventPublisher

Precondiciones: Usuario U020 con 10 puntos. No existe una solicitud REQ-404. Cola vacia.

Entradas: redeemReward("REQ-404", "U020", 25)

Paso	Descripcion
1	El servicio busca la solicitud REQ-404 en el repositorio.
2	El repositorio retorna null.
3	El flujo termina como Result.failure antes de modificar puntos o publicar eventos.

Resultados esperados: La operacion falla con mensaje "Solicitud no encontrada". El usuario mantiene 10 puntos y la cola no recibe eventos.

Resultado de los test de integracion

Las pruebas se ejecutaron localmente en JVM con Gradle, sin interfaz grafica ni emuladores.

Suite	Tests	Fallos	Errores	Omitidos	Resultado
Integracion	15	0	0	0	PASSED
JVM completa	67	0	0	0	BUILD SUCCESSFUL

Comandos de ejecucion

```
powershell -ExecutionPolicy Bypass -File .\scripts\run-integration-tests.ps1
.\gradlew.bat :app:testDebugUnitTest --tests com.ecosense.integration.EcoSenseIntegrationSpec
--rerun-tasks
.\gradlew.bat :app:testDebugUnitTest
```

Salida relevante

```
<testsuite name="com.ecosense.integration.EcoSenseIntegrationSpec" tests="15" skipped="0"
failures="0" errors="0" timestamp="2026-05-27T05:25:13.592Z" time="0.1">
EcoSenseIntegrationSpec ... IT-01 a IT-08 PASSED
BUILD SUCCESSFUL in 5s
```

Conclusion

El conjunto cubre cuatro flujos criticos del dominio EcoSense y ocho casos de integracion. Cada flujo valida un camino correcto y un camino de error, comprobando estado persistido y efectos laterales.